

PROCESO DE GESTIÓN DE FORMACIÓN PROFESIONAL INTEGRAL

FORMATO GUÍA DE APRENDIZAJE

1. IDENTIFICACIÓN DE LA GUÍA DE APRENDIZAJE

- **Denominación del Programa de Formación:** Programación Intermedia en Python (Funciones, POO y Resiliencia).
- **Competencia:** Construcción de algoritmos y modelado de arquitecturas de software.
- **Resultados de Aprendizaje:** 1. Modularizar la lógica del sistema mediante el uso de funciones puras y tipado estático (Type Hints). 2. Comprender el alcance (scope) de las variables y la pila de ejecución (call stack). 3. Abstraer entidades del mundo real utilizando Programación Orientada a Objetos (Clases). 4. Encapsular el estado interno de los objetos para proteger la integridad de los datos. 5. Reutilizar arquitecturas mediante la aplicación de Herencia y Polimorfismo. 6. Implementar resiliencia arquitectónica mediante el manejo y captura de excepciones (Try/Except).
- **Duración de la Guía:** 80 horas.

2. PRESENTACIÓN

De Operario a Arquitecto En el nivel anterior (Nivel 1), aprendiste a darle instrucciones secuenciales a la computadora. Construiste condicionales y bucles. Sin embargo, en el mundo real, los sistemas empresariales tienen millones de líneas de código. Si escribes todo en un solo bloque secuencial, el sistema colapsará, será imposible de leer y un infierno de mantener.

Ha llegado el momento de subir de nivel. En esta guía aprenderás a **Modularizar** (dividir un problema gigante en piezas pequeñas llamadas Funciones), a **Modelar** (crear planos de entidades reales usando Clases y Objetos) y a **Prevenir Fallos** (Manejo de Excepciones).

Además, adoptaremos estándares de la industria como el **Type Hinting** (declaración de tipos de datos) para que tu código deje de ser un simple "script" y se convierta en **Software de Producción**.

3. FORMULACIÓN DE LAS ACTIVIDADES DE APRENDIZAJE

FASE 1: Subcontratando el Trabajo (Funciones Básicas y Type Hinting)

Concepto: Una función es un bloque de código empaquetado y con nombre. En lugar de escribir la misma fórmula matemática diez veces, la escribes una vez dentro de una función, y simplemente "llamas" a esa función cuando la necesitas.

Explicación Detallada (El Por Qué Técnico): A nivel de CPU, llamar a una función implica un salto en la memoria. El "Call Stack" (Pila de llamadas) pausa la ejecución actual, guarda la línea donde se quedó, y salta al bloque de memoria donde vive la función. *El Upgrade Arquitectónico (Type Hints):* A partir de ahora usaremos : tipo para las entradas y -> tipo para las salidas. Esto no cambia cómo corre el programa, pero le permite a tu editor (como VS Code) avisarte de errores antes de que sucedan, comportándose como un lenguaje fuertemente tipado (C++ o Java).

Ejemplo Guiado 1.1: Creación y llamada de una función tipada (01_funciones.py)

```
# 1. Definición: '-> None' indica que esta función no devuelve datos a la memoria
def saludar_usuario() -> None:
    """Imprime un mensaje de conexión estándar."""
    print("Iniciando conexión con el servidor...")
    print("✅ Autenticación exitosa. ¡Bienvenido!")

# 2. Llamada (La ejecución): Pedirle a la función que haga su trabajo
print("--- Sistema Principal ---")
saludar_usuario()
saludar_usuario()
```

Ejemplo Guiado 1.2: Enviando datos a la función (Parámetros y Argumentos)

```
# Exigimos estrictamente que 'nombre' y 'rol' sean de tipo String (str)
def crear_perfil(nombre: str, rol: str) -> None:
    """Registra un nuevo perfil en el sistema."""
    print(f"Registrando en base de datos: {nombre} | Permisos: {rol}")

# Si enviáramos crear_perfil(123, True), el editor nos marcaría una advertencia
crear_perfil("Carlos", "Admin")
crear_perfil("Ana", "Ventas")
```

Ejemplo Guiado 1.3: La función entrega un resultado (return)

```
# Exigimos un Float (decimal) de entrada y garantizamos un Float de salida.
def calcular_iva(precio_base: float) -> float:
    """Calcula y retorna el precio total con un IVA del 19%."""
    iva: float = precio_base * 0.19
    precio_final: float = precio_base + iva
    return precio_final # Expulsa el valor hacia el Call Stack

# El resultado DEBE guardarse en una variable
factura_1: float = calcular_iva(10000.0)
factura_2: float = calcular_iva(50000.0)
```

```
print(f"Total a pagar factura 1: ${factura_1}")
print(f"Total a pagar factura 2: ${factura_2}")
```

Ejercicios Prácticos Fase 1:

1. Crea una función tipada llamada `calcular_area_rectangulo` que reciba `base: float` y `altura: float`, y retorne (`-> float`) la multiplicación de ambos. Llama la función e imprime el resultado.
2. Crea una función `es_mayor_de_edad` que reciba una `edad: int` y retorne un booleano (`-> bool`). Si la edad es `>= 18`, debe retornar `True`, de lo contrario, `False`. Prueba la función dentro de un `if`.

FASE 2: La regla de Las Vegas (Ámbito de Variables - Scope)

Concepto: Lo que pasa dentro de una función, se queda en la función. Las variables creadas dentro de un `def` mueren en cuanto la función termina. A esto se le llama **Scope local**.

Explicación Detallada (El Por Qué Técnico): Cuando una función se ejecuta, el Sistema Operativo le asigna un "Contexto de Memoria Temporal" (Stack frame). Todas las variables que crees ahí dentro nacen y viven exclusivamente en ese espacio aislado. En el instante en que la función ejecuta el `return`, el Garbage Collector de Python destruye todo ese contexto de memoria para liberar la RAM.

Ejemplo Guiado 2.1: Local vs Global (`02_scope.py`)

```
# 1. Variable GLOBAL (Vive en el programa principal, accesible por todos)
impuesto_nacional: float = 0.19

def calcular_descuento(precio: float) -> float:
    # 2. Variable LOCAL (Vive SOLO durante este bloque de memoria)
    descuento_local: float = precio * 0.10

    # La función SÍ puede leer variables Globales (hacia afuera)
    precio_con_impuesto: float = precio + (precio * impuesto_nacional)

    return precio_con_impuesto - descuento_local

resultado: float = calcular_descuento(1000.0)

# 3. ERROR: Si intentas hacer print(descuento_local) aquí afuera, el programa col
# La variable 'descuento_local' ya fue destruida de la memoria RAM.
print(f"Precio final: {resultado}")
```

Ejemplo Guiado 2.2: Valores por defecto en funciones

```
# Si el usuario no envía el parámetro 'moneda', usamos "COP" por defecto
def convertir_moneda(cantidad: float, moneda: str = "COP") -> None:
    print(f"Procesando transacción: {cantidad} en moneda {moneda}")

convertir_moneda(5000.0, "USD") # Reemplaza el valor por defecto
convertir_moneda(15000.0)      # Como no enviamos moneda, usa "COP"
```

Ejercicios Prácticos Fase 2:

1. 🌀 [EVIDENCIA DE DESEMPEÑO 1] - **Calculadora de Nómina Tipada:** Crea un script `02_reto_nomina.py`. Escribe una función `calcular_salario_neto(salario_base: float, bonificacion: float = 0.0) -> float` que calcule el salario final restando un 8% de salud y pensión, y sumando la bonificación. La bonificación debe ser cero por defecto. *El instructor validará en vivo tu comprensión del 'return', el tipado y el scope local.*

FASE 3: El Plano Arquitectónico (Introducción a Clases y Objetos)

Concepto: Las variables sueltas ya no son suficientes. Si queremos representar un "Usuario", necesitamos agrupar su nombre, edad y correo en una sola entidad. Una **Clase** es el plano o molde geométrico. Un **Objeto** es la casa física construida a partir de ese plano.

Explicación Detallada (El Por Qué Técnico): Cuando creas un objeto a partir de una clase (Instanciación), la computadora reserva un bloque pesado en la memoria RAM que contiene, agrupadamente, todas las variables (Atributos) exclusivas de esa pieza. ¿Qué es verdaderamente `self`? Es un localizador de memoria. Si instancias un objeto, este puede vivir en la dirección de RAM `0x00A1B2`. Python inyecta esa dirección automáticamente en la palabra `self` para que la clase sepa exactamente a qué bloque de memoria ir a modificarle los datos, sin afectar a los demás.

Ejemplo Guiado 3.1: Creando el molde y construyendo la casa (`03_clases.py`)

```
# 1. EL MOLDE: La Clase (Por convención, empieza con Mayúscula)
class Servidor:

    # 2. EL CONSTRUCTOR: Se ejecuta automáticamente al instanciar.
    def __init__(self, nombre: str, ip: str, ram_gb: int):
        # 3. ATRIBUTOS: Variables que le pertenecen solo a este objeto en la RAM
        self.nombre: str = nombre
        self.ip: str = ip
        self.ram: int = ram_gb
        self.estado: str = "Apagado" # Valor inicial por defecto

# 4. INSTANCIACIÓN: Construyendo los objetos físicos en RAM
server_ventas = Servidor("Ventas-01", "192.168.1.10", 16)
```

```
server_bd = Servidor("Database-Main", "10.0.0.5", 64)

# 5. Accediendo a la información específica usando el puntero del objeto
print(f"El servidor {server_ventas.nombre} tiene {server_ventas.ram}GB de RAM.")
print(f"El servidor {server_bd.nombre} actualmente está: {server_bd.estado}")
```

Ejercicios Prácticos Fase 3:

1. Crea una clase `Vehiculo` con el constructor `__init__`. Debe recibir `marca: str`, `modelo: str` y `año: int`. Crea dos objetos diferentes e imprime sus datos.

FASE 4: Acción y Comportamiento (Métodos)

Concepto: Los objetos no solo guardan datos (Atributos), también pueden realizar acciones. Una función que vive *dentro* de una clase se llama **Método**.

Explicación Detallada (El Por Qué Técnico): Cuando haces `server_ventas.encender()`, la CPU hace una traducción interna invisible: `Servidor.encender(server_ventas)`. Toma el objeto entero y lo inyecta como primer argumento. Por eso, **TODOS** los métodos de instancia deben recibir obligatoriamente `self` como primer parámetro en su definición.

Ejemplo Guiado 4.1: Dándole vida a los objetos (04_metodos.py)

```
class CuentaBancaria:
    def __init__(self, titular: str, saldo_inicial: float):
        self.titular: str = titular
        self.saldo: float = saldo_inicial

    # MÉTODO: Una acción. Nota cómo recibe 'self' y luego los parámetros útiles.
    def depositar(self, cantidad: float) -> None:
        self.saldo += cantidad
        print(f"✅ Depósito exitoso. Nuevo saldo de {self.titular}: ${self.saldo}")

    def retirar(self, cantidad: float) -> None:
        if cantidad > self.saldo:
            print(f"❌ Error: {self.titular} no tiene fondos suficientes.")
        else:
            self.saldo -= cantidad
            print(f"💰 Retiro exitoso. Saldo restante: ${self.saldo}")

# Creando y manipulando objetos
cuenta_ana = CuentaBancaria("Ana López", 50000.0)

cuenta_ana.depositar(20000.0)
cuenta_ana.retirar(100000.0) # Debería arrojar el error de fondos
cuenta_ana.retirar(15000.0)
```

Ejercicios Prácticos Fase 4:

1. 🌀 [EVIDENCIA DE DESEMPEÑO 2] - El Tamagotchi Digital: Escribe un script `04_reto_mascota.py`. Crea una clase `MascotaVirtual`. Su constructor debe recibir un `nombre: str` y arrancar con un atributo `energia: int = 10`. Crea un método `jugar(self) -> None` que reste 3 de energía y un método `dormir(self) -> None` que sume 5 de energía. Ambos métodos deben imprimir el estado actual. Instancia una mascota y hazla jugar y dormir. *El instructor evaluará tu correcta comprensión del modificador `self`.*

FASE 5: Protegiendo la Integridad (Encapsulamiento)

Concepto: En la Fase 4, nada impide que alguien escriba `cuenta_ana.saldo = -5000000` y arruine el sistema. El **Encapsulamiento** es esconder las variables críticas en la memoria, obligando a los programadores a modificarlas únicamente a través de Métodos controlados (que tienen validaciones lógicas).

Explicación Detallada (El Por Qué Técnico): A diferencia de Java, Python no tiene barreras de memoria estrictas (`private` no existe a nivel compilador). Si a un atributo le pones un guion bajo `_` al inicio (ej. `self._saldo`), es una convención para decir "No tocar". Pero si le pones doble guion bajo `__`, Python activa un algoritmo llamado *Name Mangling* que renombra la variable a nivel de RAM (ej. `_SistemaSeguridad__pin_secreto`), haciéndola casi invisible e inaccesible desde el exterior del objeto.

Ejemplo Guiado 5.1: Name Mangling en acción (`05_seguridad.py`)

```
class SistemaSeguridad:
    def __init__(self, pin_acceso: int):
        self.usuario: str = "Admin" # Público
        # Atributo Privado (Fuerte) usando doble guion bajo
        self.__pin_secreto: int = pin_acceso
        self.__alarma_activada: bool = True

    # Método "Setter" (Puerta de acceso controlada)
    def desactivar_alarma(self, intento_pin: int) -> None:
        if intento_pin == self.__pin_secreto:
            self.__alarma_activada = False
            print("🔒 Alarma desactivada.")
        else:
            print("🚨 INTRUSO DETECTADO. Llamando a la policía.")

casa_central = SistemaSeguridad(1234)

# print(casa_central.__pin_secreto) -> ESTO ARROJARÁ UN ERROR DE ATRIBUTO
casa_central.desactivar_alarma(9999) # Rechazado por la lógica del método
casa_central.desactivar_alarma(1234) # Aceptado
```

FASE 6: Reutilizando Arquitectura (Herencia)

Concepto: Si tenemos una clase `Perro` y una clase `Gato`, ambas tienen `nombre` y ambas pueden `comer()`. En vez de escribir el mismo código dos veces, creamos una "Clase Padre" llamada `Animal` y hacemos que `Perro` y `Gato` hereden de ella, ahorrando cientos de líneas de código.

Explicación Detallada (El Por Qué Técnico): La herencia conecta las tablas de métodos de dos clases. Si llamas a un método en la clase "Hijo" y la CPU no lo encuentra en su bloque de RAM, el puntero automáticamente sube a buscarlo en el espacio de memoria de la clase "Padre". Si el Hijo crea un método con el mismo nombre que el Padre, se sobrescribe en memoria (Polimorfismo).

Ejemplo Guiado 6.1: Padres e Hijos (06_herencia.py)

```
# 1. CLASE PADRE (Superclase)
class Empleado:
    def __init__(self, nombre: str, salario: float):
        self.nombre: str = nombre
        self.salario: float = salario

    def trabajar(self) -> None:
        print(f"{self.nombre} está cumpliendo su horario regular.")

# 2. CLASE HIJO (Subclase) - Hereda todo de Empleado pasándolo por paréntesis
class Desarrollador(Empleado):
    def programar(self) -> None:
        print(f"💻 {self.nombre} está escribiendo código Python.")

# 3. CLASE HIJO CON POLIMORFISMO
class Gerente(Empleado):
    # Sobrescribe el método del Padre
    def trabajar(self) -> None:
        print(f"🗣️ {self.nombre} está en una reunión estratégica.")

# Pruebas de campo
dev = Desarrollador("Carlos", 3500.0)
jefe = Gerente("Ana", 6000.0)

dev.trabajar()    # Sube a RAM y usa el método del Padre
dev.programar()  # Usa su método propio

jefe.trabajar()  # Usa su propio método sobrescrito (Polimorfismo)
```

FASE 7: Resiliencia Arquitectónica (Try / Except)

Concepto: Un usuario *siempre* se va a equivocar. Si le pides un número, ingresará texto. Si tu programa simplemente se cierra y colapsa (Crash), has fallado como arquitecto. Debemos

"capturar" el error, aislarlo, y mantener el programa funcionando.

Explicación Detallada (El Por Qué Técnico): Cuando el intérprete de Python detecta una operación ilegal en la CPU (como dividir por cero o convertir la letra "A" en un entero), genera una señal de interrupción llamada "Excepción". Si tu código no tiene una red de seguridad (`try`), esta señal sube por todo el *Call Stack* hasta destruir el proceso principal, cerrando la aplicación. El bloque `try/except` actúa como un amortiguador que absorbe el impacto en memoria.

Ejemplo Guiado 7.1: La red de seguridad (`07_excepciones.py`)

```
class CajeroAutomatico:
    def __init__(self):
        self.efectivo_disponible: float = 10000.0

    def solicitar_retiro(self) -> None:
        print("--- Bienvenido al Cajero ---")

        # Bloque TRY: "Intenta hacer esto, pero estate alerta"
        try:
            monto_str: str = input("Ingrese la cantidad a retirar (solo números):")
            # Si el usuario ingresa letras, esta conversión generará un ValueError
            monto: float = float(monto_str)

            # Si el usuario ingresa cero, lanzaremos nuestra propia excepción lógica
            if monto == 0:
                raise ValueError("No puede retirar cero pesos.")

            # Operación matemática (riesgo de dividir por cero, etc.)
            self.efectivo_disponible -= monto
            print(f"Retiro exitoso. Quedan ${self.efectivo_disponible} en el cajero")

        # Bloque EXCEPT: "Si ocurre este error específico, no te apagues, haz esto"
        except ValueError as e:
            print(f"⚠️ ERROR DE FORMATO: Usted ingresó caracteres inválidos. ({e})")

        # Un Exception general captura cualquier otro error inesperado (como un FileNotFoundError)
        except Exception as e:
            print(f"🔥 ERROR CRÍTICO DESCONOCIDO: Contacte soporte. Detalles: {e}")

        # Bloque FINALLY: "Se ejecuta SIEMPRE, haya ocurrido un error o no"
        finally:
            print("Expulsando tarjeta... Gracias por utilizar nuestra red.\n")

# Prueba
mi_cajero = CajeroAutomatico()
# Intenta ingresar letras como "Hola" en vez de números para probar la resiliencia
mi_cajero.solicitar_retiro()
```


Ejercicios Prácticos Fases 6 y 7:

1.  **[EVIDENCIA DE PRODUCTO FINAL] - Sistema Resiliente de Inventario:** Crea un archivo `08_producto_inventario.py`.
 - Crea una clase `Producto` con `__init__` que reciba `nombre: str`, `precio: float` y `stock: int`.
 - Añade el método `vender(self, cantidad: int) -> None`.
 - **Requisito de Resiliencia:** Dentro de `vender`, usa un bloque `try/except`. Si el usuario pide comprar más cantidad del stock disponible, debes levantar un error intencional (`raise ValueError("Stock insuficiente")`) y el `except` debe imprimir la advertencia sin que el programa colapse.
 - Crea una clase heredada `ProductoPerecedero` que herede de `Producto`, añadiendo un atributo `dias_vencimiento: int`.
 - Instancia productos, realiza ventas (exitosas y fallidas por falta de stock) imprimiendo los resultados. Sube este código comentado (.zip) a la plataforma LMS Zajuna.

4. PREPARACIÓN PARA LA EVIDENCIA DE CONOCIMIENTO

Para el cuestionario técnico en la plataforma LMS, domina los siguientes conceptos arquitectónicos:

1. **El modificador de contexto `self`:** Comprender de forma precisa por qué es obligatorio, y cómo permite que los métodos sepan qué bloque específico de memoria (objeto) deben alterar.
2. **Type Hinting y Prevención:** Explicar cómo el tipado estático opcional (`: tipo`) mejora la mantenibilidad del código en arquitecturas complejas.
3. **Pila de Ejecución (Scope) e Interrupciones:** Diferenciar el comportamiento de la RAM entre una variable global y una local. Además, comprender cómo una Excepción interrumpe la pila de llamadas si no es envuelta en un bloque `try`.
4. **Pilares de la POO:** Diferenciar Instanciación, Encapsulamiento (Name Mangling con `__`) y Herencia/Polimorfismo.

5. GLOSARIO TÉCNICO

- **Type Hinting:** Sintaxis introducida en Python 3.5 para anotar los tipos esperados de variables, parámetros y retornos, previniendo errores de arquitectura.
- **Call Stack (Pila de Llamadas):** Estructura de datos en RAM que lleva el registro de qué subrutina se está ejecutando en ese milisegundo.
- **Excepción:** Un evento anómalo durante la ejecución de un programa que altera su flujo normal y amenaza con colapsar el proceso si no es "capturado" (Catch/Except).

- **Instanciación:** El proceso de asignación de memoria RAM en el que un molde teórico (Clase) se convierte en datos reales (Objeto).
- **Name Mangling:** Mecanismo de seguridad en Python que altera internamente el nombre de un atributo (los que tienen `__`) para bloquear su acceso desde el exterior.

6. PLANTEAMIENTO DE EVIDENCIAS DE APRENDIZAJE PARA LA EVALUACIÓN

Nota: La evaluación se divide según el modelo pedagógico del SENA estructurado en Conocimiento, Desempeño y Producto.

Fase del proyecto formativo	Actividad del proyecto formativo	Actividad de Aprendizaje	Evidencias de Aprendizaje	Criterios de Evaluación	Técnicas e Instrumentos de Evaluación
Planeación / Ejecución	Modelar la estructura lógica y arquitectónica del sistema de información.	AP04 - Modularización y Tipado.	1. Evidencia de Conocimiento:	• Define la estructura y el comportamiento de un objeto.	Técnica: Formulación de preguntas.
		Asimilar los conceptos de subrutinas, type hinting, alcance de variables (scope) e instanciación de objetos.	Cuestionario técnico sobre gestión de memoria en funciones locales, manejo de excepciones (Try/Catch), el rol del puntero <code>self</code> y los fundamentos del modelo Orientado a Objetos (Encapsulamiento).	• Identifica el ciclo de vida de una variable local. • Identifica el rol del manejo de excepciones para la continuidad de un sistema.	Instrumento: Cuestionario (LMS Zajuna).

Planeación / Ejecución	Modelar la estructura lógica y arquitectónica del sistema de información.	AP05 – Comportamiento de Entidades. Desarrollar y validar lógica de negocio a través de funciones tipadas y métodos de clase.	2. Evidencia de Desempeño: Participación y observación directa del aprendiz codificando en vivo la Calculadora de Nómina Tipada (Fase 2) y El Tamagotchi Digital (Fase 4 - Instanciación).	• Modulariza la solución al problema implementando funciones con retornos explícitos. • Gestiona el estado del objeto manipulando atributos mediante <code>self</code>.	Técnica: Observación Sistemática y Code Review iterativo. Instrumento: Rúbrica de Desempeño Lógico.
Planeación / Ejecución	Modelar la estructura lógica y arquitectónica del sistema de	AP06 – Arquitecturas Resilientes.	3. Evidencia de Producto:	• Transforma requerimientos del mundo real en componentes	Técnica: Valoración de Producto.